

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший разработчик в X5 Tech

_____ В. Е. Козловский

« ____ » _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия", старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев

« ____ » _____ 2026 г.

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.09 81 01-1-ЛУ

Исполнители:

Студент группы БПИ236

_____ / С. Е. Растворов /

« ____ » _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл	

УТВЕРЖДЕН

RU.17701729.05.09 81 01-1-ЛУ

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Пояснительная записка

RU.17701729.05.09 81 01-1

Листов 16

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Пояснительная записка содержит сведения о назначении программы, области ее применения, технических характеристиках, выбранных средствах реализации, архитектурных решениях и ожидаемых технико-экономических показателях разработки.

Документ подготовлен в составе программной документации курсового проекта «Разработка интеллектуального ассистента для образовательных учреждений» в соответствии с требованиями Единой системы программной документации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Документ(ы), на основании которого(ых) ведется разработка	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	6
3.1. Постановка задачи на разработку программы	6
3.2. Описание и обоснование выбора состава технических и программных средств	6
3.2.1. Состав технических и программных средств	6
3.2.2. Обоснование выбора технических и программных средств	6
3.3. Описание и обоснование архитектуры программы	7
3.4. Особенности функционала	8
3.4.1. Основной сценарий работы	8
3.4.2. Лексический резервный режим	8
3.4.3. Загрузка и индексация документов	8
3.4.4. История диалогов	8
3.4.5. Контур развёртывания	8
3.5. Обоснование выбора метода организации входных и выходных данных	8
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	10
4.1. Ориентировочная экономическая эффективность	10
4.2. Предполагаемая потребность	10
4.3. Экономические преимущества разработки по сравнению с аналогами	10
ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ	12
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЪЗУЕМОЙ ЛИТЕРАТУРЫ	13
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ	14
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ	15

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ**1.1. Наименование программы**

Наименование программы – «Разработка интеллектуального ассистента для образовательных учреждений».

Наименование программы на английском языке – «Development of an Intelligent Assistant for Educational Institutions».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Программа «Разработка интеллектуального ассистента для образовательных учреждений» предназначена для интеллектуальной поддержки студентов, преподавателей и сотрудников учебных офисов НИУ ВШЭ и других вузов: автоматизированные ответы на вопросы об учебных планах, регламентах, расписаниях и сервисах на основе внутренних документов учреждения.

Функциональное назначение включает следующие задачи: семантический поиск релевантных фрагментов внутренних документов по вопросу пользователя; генерация ответа с явным цитированием источников и указанием страницы или ссылки; поддержка диалоговой истории для уточнения вопросов; фоновое обновление корпуса документов без прерывания сервиса; деградация до лексического поиска при недоступности LLM; сбор метрик качества; разграничение доступа по ролям пользователь/администратор.

2.2. Эксплуатационное назначение

Программа предназначена для эксплуатации в составе контейнеризованного стека Docker Compose, разворачиваемого на серверах учебного учреждения. Штатная работа предполагает: доступный экземпляр Ollama с моделью Mistral для генерации ответов; ChromaDB с персистентным векторным индексом на диске; PostgreSQL 16 для хранения истории диалогов и данных пользователей; Telegram-бот и Streamlit-клиент как клиентские интерфейсы.

Система рассчитана на пилотную фокус-группу 50—100 активных пользователей с нагрузкой до 300 запросов в сутки и доступностью не менее 95% в рабочее время 09:00—21:00 МСК. Специальной подготовки рабочего места пользователя не требуется: для работы через Telegram — стандартный клиент мессенджера; для работы через веб — современный браузер.

2.3. Краткая характеристика области применения

Область применения программы — информационное обслуживание участников образовательного процесса. Система ориентирована на сценарии, в которых пользователь задаёт конкретный вопрос по документам вуза и получает ответ с указанием источника, не прибегая к ручному поиску по PDF-файлам или веб-страницам.

С точки зрения состава пользователей система охватывает три группы: студентов бакалавриата и магистратуры (учебные планы, сроки, процедуры), преподавателей и учебных офисов (регламенты, сервисы), ИТ-служб и методистов (быстрые справки). Программа дополняет официальные FAQ-разделы сайтов вузов, однако, в отличие от них, опирается на внутренние документы конкретного учреждения и даёт ответы с точными ссылками на источники.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Программа «Разработка интеллектуального ассистента для образовательных учреждений» предназначена для ответа на вопросы пользователей по внутренним документам образовательного учреждения с использованием архитектуры RAG (Retrieval-Augmented Generation). Система должна принимать текстовый вопрос, находить релевантные фрагменты в векторной базе документов и генерировать ответ с явным цитированием источников.

К программе предъявляются следующие основные ограничения: разграничение доступа по токену с ролевой моделью; поддержка режима работы без внешних LLM-сервисов (локальный Ollama); устойчивость при недоступности LLM через лексический fallback; фоновое обновление индекса без остановки сервиса; контейнерное развёртывание без зависимости от конкретной ОС.

Ожидаемые результаты работы: не менее 85% контрольных вопросов получают корректный ответ по документам; среднее время ответа POST /ask не превышает 7 секунд при доступном Ollama; ответ содержит ссылки на источники; лексический fallback работает при недоступном Ollama; сервис доступен не менее 95% рабочего времени (09:00—21:00 МСК).

3.2. Описание и обоснование выбора состава технических и программных средств

3.2.1. Состав технических и программных средств

Серверная часть разворачивается в контейнерной среде Docker Compose. В состав программного стека входят:

- Python 3.12 — основной язык разработки;
- FastAPI — HTTP-фреймворк RAG-сервера;
- LangChain — оркестрация RAG-пайплайна;
- ChromaDB — персистентное векторное хранилище;
- HuggingFace Transformers, модель ruBERT-tiny2 — вычисление эмбеддингов;
- Ollama (Mistral) — локальная LLM для генерации ответов;
- PostgreSQL 16 / SQLAlchemy (asyncpg) — хранение истории диалогов и пользователей;
- Tesseract OCR, PyPDF2 — извлечение текста из PDF и изображений;
- aiogram 3.x — Telegram-бот;
- Streamlit — веб-клиент;
- Docker и Docker Compose — контейнеризация и оркестрация.

Для управления зависимостями используется uv; для линтинга — ruff; для тестирования — pytest.

3.2.2. Обоснование выбора технических и программных средств

FastAPI выбран как основа серверной части, поскольку обеспечивает нативную поддержку asyncio, автоматическую генерацию документации Swagger и строгую валидацию запросов через Pydantic. Это критично для высоконагруженного диалогового эндпоинта POST /ask.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

LangChain использован как оркестратор RAG-цепочки: он абстрагирует взаимодействие с разными LLM и векторными хранилищами, упрощая переключение между Ollama/Mistral и другими провайдерами без переписывания бизнес-логики.

ChromaDB выбран как векторное хранилище с персистентным хранением на диске без необходимости отдельного сервера. Это снижает сложность развёртывания и упрощает пересборку индекса.

ruBERT-tiny2 выбран как лёгкая модель эмбеддингов, оптимизированная для русскоязычных текстов при умеренных требованиях к CPU/RAM, что критично для локального развёртывания в вузе.

Ollama/Mistral выбран для обеспечения полностью локальной генерации ответов без зависимости от внешних API и передачи данных за периметр сети учебного учреждения.

PostgreSQL 16 используется для хранения истории диалогов и пользовательских данных, поскольку система требует транзакционной согласованности при записи сессий и аутентификационных токенов.

Tesseract OCR применён для извлечения текста из изображений и сканов документов, что позволяет включать в базу знаний материалы, недоступные в электронном текстовом формате.

aiogram 3.x выбран как актуальная асинхронная библиотека для Telegram-ботов с поддержкой Finite State Machine и middleware.

Docker Compose обеспечивает воспроизводимое развёртывание полного стека на любом Linux-сервере без зависимости от локальной конфигурации среды.

3.3. Описание и обоснование архитектуры программы

Архитектура системы построена вокруг FastAPI RAG-сервера как центрального компонента. Клиентские интерфейсы (Telegram-бот и Streamlit-клиент) взаимодействуют с сервером исключительно через HTTP-эндпоинты, что обеспечивает их независимость и заменяемость.

RAG-пайплайн (src/server/app/rag.py) реализован следующим образом: запрос поступает через POST /ask; модуль vector.py вычисляет эмбеддинг вопроса моделью ruBERT-tiny2 и выполняет поиск в ChromaDB по косинусному сходству; top-k найденных фрагментов передаются в LLM через LangChain; ответ LLM с указанием источников возвращается клиенту.

Модуль lexical.py реализует резервный поиск и активируется автоматически при недоступности Ollama, обеспечивая graceful degradation без остановки сервиса.

Модуль document_ingestion.py отвечает за конвейер загрузки: чтение PDF (PyPDF2), HTML, TXT; OCR изображений (Tesseract); нормализация; разбивка на чанки до 500 символов с перекрытием. Модуль index_documents.py запускает индексацию как фоновую задачу через POST /index.

Модуль conversation_memory.py сохраняет историю диалогов в PostgreSQL через SQLAlchemy (asyncpg). Модуль metrics.py накапливает время ответа, confidence score и флаги эскалации.

Аутентификация (auth_*.py) реализована через Bearer-токен с ролевой моделью пользователь/администратор.

Клиентский уровень: Telegram-бот (src/bot/) принимает сообщения через aiogram 3.x и передаёт вопрос в POST /ask; Streamlit-клиент (src/client/app/streamlit_app.py) предоставляет диалоговый веб-интерфейс.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.4. Особенности функционала

3.4.1. Основной сценарий работы

Основной сценарий: пользователь задаёт вопрос через Telegram-бот или Streamlit; вопрос передаётся в POST /ask; RAG-пайплайн ищет релевантные фрагменты в ChromaDB; LLM генерирует ответ с цитированием источников; ответ с перечнем документов-источников возвращается пользователю. Среднее время выполнения сценария при доступном Ollama — до 7 секунд.

3.4.2. Лексический резервный режим

При недоступности Ollama сервис переключается в лексический режим: модуль lexical.py выполняет поиск в проиндексированных документах и возвращает релевантные фрагменты как ответ. Ответ в этом режиме содержит флаг fallback: true. Данный режим предназначен для сохранения работоспособности при технических сбоях LLM и не является приёмочным критерием качества ответа.

3.4.3. Загрузка и индексация документов

Администратор загружает документы (PDF, HTML, TXT, изображения) в каталог корпуса и запускает POST /index. Модуль document_ingestion.py извлекает текст, нормализует и разбивает на чанки; модуль index_documents.py вычисляет эмбединги и сохраняет их в ChromaDB. Операция выполняется в фоне и не прерывает обработку запросов пользователей.

3.4.4. История диалогов

Система сохраняет историю вопросов и ответов каждого пользователя в PostgreSQL. При последующих запросах история может использоваться для построения контекста, позволяя пользователю задавать уточняющие вопросы без повторения исходного контекста.

3.4.5. Контур развёртывания

Локальный стенд запускается командой docker compose up -d с использованием docker-compose.yaml. Production-контур описан в deployment/production/compose.yaml. Переменные окружения задаются через .env-файл: BOT_TOKEN, DATABASE_URL, LLM_HOST, CHROMA_PATH, ALLOW_LLM_FALLBACK. Healthcheck-эндпоинт GET /health используется для проверки готовности сервиса.

3.5. Обоснование выбора метода организации входных и выходных данных

Входные данные программы разделены на три потока: пользовательские запросы (текстовые вопросы через API, Telegram или Streamlit), документы для индексации (PDF, HTML, TXT, PNG/JPG) и параметры конфигурации (переменные окружения). Такое разделение позволяет независимо обновлять корпус документов без перезапуска сервиса и обеспечивает чёткое разграничение ответственности между компонентами.

Выходные данные возвращаются в JSON-формате через REST API: ответ содержит текст, источники, confidence и флаг fallback. Такой формат обеспечивает единообразное потребление как Telegram-ботом, так и Streamlit-клиентом.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Внутреннее хранение разделено по технологиям: векторные эмбединги и метаданные чанков — в ChromaDB на диске; история диалогов и токены — в PostgreSQL. Разделение обеспечивает независимость жизненных циклов векторного индекса и транзакционных данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Разработка системы «Разработка интеллектуального ассистента для образовательных учреждений» не ориентирована на прямую коммерческую выручку. Ожидаемый экономический эффект выражается в снижении трудоёмкости информационного обслуживания студентов и преподавателей: автоматизированные ответы на типовые вопросы по учебным планам, срокам, регламентам и сервисам вуза снижают нагрузку на учебные офисы и справочные службы.

Дополнительный эффект достигается за счёт следующих факторов:

- сокращение времени ожидания ответа для пользователя с нескольких часов (при ручном обращении) до нескольких секунд (автоматический RAG-ответ);
- уменьшение числа повторных обращений по одинаковым вопросам благодаря истории диалогов;
- снижение затрат на эксплуатацию за счёт полностью локального развёртывания без облачных LLM-сервисов;
- возможность масштабирования на несколько факультетов или программ без изменения кода путём обновления корпуса документов.

Для учебного подразделения эффект выражается в сокращении операционной нагрузки на справочные службы и повышении доступности информации для студентов в любое время суток.

4.2. Предполагаемая потребность

Потребность в системе обусловлена значительным объёмом повторяющихся вопросов студентов по регламентам, срокам и процедурам. Пилотная аудитория системы — 50—100 активных пользователей с нагрузкой до 300 запросов в сутки — соответствует типичному масштабу одной образовательной программы.

Разработка востребована следующими группами пользователей:

- студентами бакалавриата и магистратуры, которым требуются быстрые ответы по учебным планам и регламентам;
- преподавателями и учебными офисами для быстрых справок по процедурам;
- ИТ-службами и методистами для автоматизации информационной поддержки.

По мере роста корпуса документов и числа пользователей система может быть масштабирована горизонтально (несколько экземпляров бота) и путём вынесения Ollama на отдельный GPU-узел.

4.3. Экономические преимущества разработки по сравнению с аналогами

Существующие аналоги — официальные FAQ-разделы сайтов вузов (hse.ru/answers), ChatGPT и YandexGPT — отличаются от разрабатываемой системы по следующим ключевым параметрам: ответы опираются на внутренние документы конкретного учреждения, а не на общедоступные интернет-данные; ответы сопровождаются точными ссылками на источники; система развёртывается локально внутри периметра сети учреждения без передачи данных внешним сервисам; корпус документов обновляется без изменения кода.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Функция	Официальный FAQ вуза	ChatGPT / YandexGPT	Разработка интеллектуального ассистента для образовательных учреждений
Ответы на основе внутренних документов	-	-	+
Ссылки на источники и страницы	-	-	+
Локальное развёртывание внутри периметра	+	-	+
Обновляемый корпус документов	-	-	+
Диалоговая история	-	+	+
Лексический fallback при сбое LLM	—	—	+
Метрики качества ответов	-	-	+
Итого	1	1	7

Таблица 1. Сравнение функциональных характеристик с аналогами

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ

При подготовке данной курсовой работы генеративные модели использовались как вспомогательный инструмент при работе с текстовой документацией. Использование не носило автономного характера: итоговые формулировки, отбор фактов, проверка соответствия техническому заданию, сопоставление с исходным кодом и окончательная редактура выполнялись авторами работы.

Генеративная модель применялась для следующих целей:

- подготовки черновых формулировок отдельных фрагментов пояснительной записки, программы и методики испытаний, руководства оператора и связанной программной документации;
- унификации терминологии и стиля изложения в разных документах комплекта;
- проверки согласованности между текстом пояснительной записки, техническим заданием и фактическим составом функций проекта;
- редактирования русскоязычных технических формулировок и сокращения числа стилистических несоответствий;
- подготовки черновых вариантов отдельных фрагментов программного кода, конфигурации и служебных сценариев при последующей ручной доработке.

В работе использовалась генеративная модель Claude (Anthropic), доступная через инструментальную среду Claude Code. Адрес источника в сети «Интернет»: <https://claude.ai/>. Способ применения модели: формирование черновых вариантов текста, предложение альтернативных формулировок, редактура отдельных разделов, выявление потенциальных противоречий между документами, а также подготовка черновых фрагментов кода и конфигурации. Использование модели не означало автоматического принятия сгенерированного результата: все предложенные фрагменты программного кода, конфигурации и документации проходили ручную проверку, адаптацию к структуре проекта и последующее тестирование либо сверку с исходным кодом. Содержательные решения о составе системы, описании архитектуры, перечне функций и результатах испытаний принимались авторами работы на основании исходного кода, технического задания и сопутствующей документации проекта.

Использование генеративной модели относится не ко всей работе целиком, а к отдельным фрагментам текста и документации. Наибольший объём применения пришёлся на разделы, связанные с выравниванием стиля, согласованием терминологии и подготовкой служебной документации. Применение модели позволило ускорить подготовку черновых формулировок и техническую редактуру, однако не заменяло ручную проверку фактов, сверку с кодовой базой, контроль соответствия ТЗ и окончательное авторское редактирование.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Правила использования искусственного интеллекта студентами НИУ ВШЭ. URL: https://www.hse.ru/studyspravka/ai_guidelines/ (дата обращения: 11.05.2026).
15. Положение об организации промежуточной аттестации и текущего контроля успеваемости студентов Национального исследовательского университета «Высшая школа экономики» (ПОПАТКУС). URL: https://www.hse.ru/studyspravka/ai_guidelines/ (дата обращения: 11.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
Официальные ответы НИУ ВШЭ (FAQ)	https://www.hse.ru/answers
ChatGPT	https://chat.openai.com
YandexGPT	https://ya.ru/ai/gpt

Дата обращения: 11.05.2026.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

Термин	Определение
RAG	Retrieval-Augmented Generation — архитектура, в которой ответ LLM генерируется на основе фрагментов, извлечённых из векторной базы документов.
ChromaDB	Встраиваемое векторное хранилище с персистентным хранением на диске, используемое для семантического поиска по чанкам документов.
Эмбединг	Векторное представление текстового фрагмента, получаемое моделью ruBERT-tiny2 для последующего поиска по сходству.
Чанк	Нормализованный фрагмент текста документа длиной до 500 символов с перекрытием, единица хранения в ChromaDB.
LLM	Large Language Model — большая языковая модель (Ollama/Mistral), используемая для генерации ответа на основе контекста из найденных чанков.
Fallback	Режим работы системы при недоступности LLM: ответ формируется через лексический поиск без нейросетевой генерации.
OCR	Optical Character Recognition — извлечение текста из изображений (PNG/JPG) через Tesseract.
Confidence	Оценка уверенности модели в ответе, числовое значение в диапазоне [0, 1].
REST/JSON API	Прикладной интерфейс системы, использующий HTTP и формат JSON для обмена данными.
Ролевая модель	Механизм разграничения доступа: роли «пользователь» и «администратор» определяют допустимые операции.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]